Practical 4: Exception Handling and Fetching Data from a REST API

Aim

To study and implement exception handling in Python while fetching data from a REST API.

Theory

Exception handling in Python is done using try, except, else, and finally blocks: code that might raise a runtime error is placed in the try block, the except block catches and handles specific (or general) exception types so the program does not crash abruptly, an optional else block runs only if no exception occurred, and the finally block always executes regardless of whether an exception was raised, making it ideal for cleanup actions like closing files or connections. Catching specific exception classes (e.g., URLError, TimeoutError) instead of a bare except is considered good practice as it avoids silently hiding unrelated bugs. A REST API (Representational State Transfer) is an architectural style that allows applications to communicate over HTTP using standard methods such as GET, POST, PUT, and DELETE, typically exchanging data in JSON (JavaScript Object Notation) format because it is lightweight and language-independent. The urllib.request module (a built-in alternative to third-party libraries like requests) can be used to send HTTP requests and read server responses, while the json module provides json.loads() to parse a JSON string/bytes into native Python objects (dicts/lists) and json.dumps() to convert Python objects back into a formatted JSON string.

Algorithm

1. Import the json and urllib.request modules.
2. Define the API URL to fetch data from.
3. Use a try block to open the URL and read the response with a timeout.
4. Parse the response using json.loads() and print it in readable format using json.dumps().
5. Use an except block to catch and print any errors during the request.
6. Use a finally block to indicate that the request has completed.

Program

```
import json
import urllib.request

try:
    url = "https://jsonplaceholder.typicode.com/todos/1"
    with urllib.request.urlopen(url, timeout=5) as response:
        data = json.loads(response.read())
        print("API Response:")
        print(json.dumps(data, indent=2))
except Exception as e:
    print("Error fetching data:", e)
finally:
    print("Request completed")
```

Output

```
API Response:
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
Request completed
```

Result / Conclusion

Exception handling was successfully used to safely fetch and parse JSON data from a REST API.

Viva Questions

Q1. What is the difference between try-except-else-finally blocks?

Ans: try: code that may raise an exception. except: runs only if an exception occurred in try — handles the error. else: runs only if NO exception occurred in try — for code that should execute on success. finally: always runs regardless of whether an exception occurred or not — used for cleanup (closing files, releasing locks). 'else' and 'finally' are optional.

Q2. What are the four main HTTP methods used in REST APIs?

Ans: GET — retrieve a resource (read-only, no side effects); POST — create a new resource; PUT/PATCH — update an existing resource (PUT replaces entirely, PATCH modifies partially); DELETE — remove a resource. These map directly to CRUD operations: Create=POST, Read=GET, Update=PUT/PATCH, Delete=DELETE.

Q3. What is the difference between JSON and XML?

Ans: JSON (JavaScript Object Notation) uses key-value pairs and arrays, is more compact, human-readable, and natively supported by JavaScript. XML (eXtensible Markup Language) uses opening/closing tags, supports attributes and namespaces, and is more verbose. JSON is preferred for REST APIs due to smaller payload size and easier parsing in most languages. XML is still used in SOAP web services and configuration files.

Q4. Why should you catch specific exceptions rather than using a bare 'except:'?

Ans: A bare 'except:' catches everything including KeyboardInterrupt, SystemExit, and MemoryError — errors you usually don't want to suppress. It can hide bugs by swallowing unexpected errors silently. Catching specific exceptions (e.g., urllib.error.URLError, json.JSONDecodeError) makes the code more maintainable, easier to debug, and ensures only expected error types are handled, letting genuinely unexpected errors propagate.

Q5. What is the difference between urllib.request and the requests library?

Ans: urllib.request is part of Python's standard library (no installation needed) but has a more verbose API. The requests library is a third-party package (pip install requests) with a much simpler, more Pythonic API: requests.get(url) vs urllib.request.urlopen(url). requests also automatically handles encoding, sessions, cookies, and authentication more easily. urllib is preferred when you want zero dependencies; requests is preferred for real-world applications.